

test_gitignore_swallow_is_loud

test_gitignore_swallow_is_loud.sh

Revision: 1 Last modified: 2026-08-26T00:00:00Z

Overview

BOB-212 RED (§11.4.115 / §11.4.224). Proves that authoring a NEW credential-named **source** file under a first-party source root is silently swallowed by .gitignore, and that **no signal** distinguishes “the file was swallowed” from “no file was authored” — a §11.4.201(6) FALSE-NULL located in the commit path itself.

Prerequisites

bash, git, mktemp. No network, no containers, no credentials.

Usage

```
bash tests/security/test_gitignore_swallow_is_loud.sh          #  
    guard mode  
RED_MODE=1 bash tests/security/test_gitignore_swallow_is_loud.sh  
    # reproduction mode  
SWALLOW_GUARD=/path/to/guard bash tests/security/...         #  
    point at a candidate fix
```

Exit codes

code	meaning
0	

code	meaning
	guard mode: the swallow is LOUD (fixed). RED_MODE=1: defect reproduced.
1	guard mode: the swallow is SILENT — defect present. RED_MODE=1: not reproducible.
2	INSTRUMENT BROKEN or SECRET LEAK. Verdict void.

Exit 2 is deliberately distinct from exit 1 so a blind harness can never be read as “defect present” (§11.4.1 — a FAIL for a script-internal reason is a FAIL-bluff).

What it asserts

1. **Control needle** (§11.4.201(7)(b)) — a non-credential-named sibling in the same directory *is* visible in `git status`. If it is not, the harness is blind and every zero below is void.
2. **Acceptance disjunction** (BOB-212) — the subject file either commits normally **or** a swallow-guard refuses loudly, naming the file *and* the `.gitignore:<line>` rule. Today neither holds.
3. **The silence** — `git status --porcelain` is captured in two scratch worlds, one where the file was authored and one where it never existed. Today the two outputs are **byte-identical**. That identity is the false-null.
4. **Golden-FALSE** (§11.4.201(1) / §11.4.10) — 15 secret-bearing shapes (`.env`, `*.pem`, `*.key`, `*.p12`, `*.jks`, `*_creds.json`, `*credentials*.yaml`, `cookies_*.txt`, `RUTRACKER_*`, `extensionless service_credentials`, ...) must remain ignored. Without this, a “fix” that simply deletes the glob looks green.

Why it is fix-direction-agnostic

The direction is an operator decision (§11.4.66), so the test asserts the *disjunction* rather than a mechanism. Verified both ways (see `docs/qa/BOB-212/red_run_polarity_flip.log`): it flips GREEN under a narrowed glob **and** under a swallow-guard stand-in.

Anti-bluff provenance

run	result
guard mode, today	exit 1 — RED, false-null confirmed
RED_MODE=1, today	exit 0 — reproduction captured
with guard stand-in (direction b)	exit 0 — flips GREEN
with narrowed glob (direction a)	exit 0 — flips GREEN
§1.1 mutation: delete the globs (naive fix)	exit 2 — 11 leaks caught

The last row is what proves the golden-FALSE is load-bearing rather than decorative.

Side effects

Creates and removes one `mktemp -d` scratch tree. It copies the repository's own `.gitignore` into that tree, so it tests the **real** rules (§11.4.27, no mock). It never writes to, stages, adds, or modifies anything in the repository.

Related

- docs/qa/B0B-212/blast_radius_1a_tracked_ignored.txt
- docs/qa/B0B-212/blast_radius_1b_untracked_swallowed.txt
- docs/qa/B0B-212/rule_evaluation.txt
- docs/qa/B0B-212/red_run.log, red_run_polarity_flip.log

Last verified: 2026-08-26